Model Context Protocol (MCP): Avaliação da Qualidade de Descrições de Tools com Base em Contexto de Código Fonte

Pedro H. P. Rodrigues[1], **Vinicius R. A. de A. Moreira**[1]

[1]Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais
Caixa Postal 500 – 30535-901 – Belo Horizonte – MG – Brazil

viniciusarantes133@gmail.com, pedrohenriquepr08@gmail.com

Resumo. *O Model Context Protocol (MCP) permite que agentes de Inteligência Artificial (IA) interajam com sistemas externos por meio de ferramentas (tools), cujas descrições em linguagem natural orientam sua utilização. Descrições de baixa qualidade, caracterizadas pela presença de smells, podem comprometer a eficácia desses agentes. Este trabalho investiga se a avaliação da qualidade dessas descrições muda quando o avaliador tem acesso ao código da ferramenta. Para isso, 12.166 ferramentas de 206 repositórios de servidores MCP do GitHub são extraídas, junto com o código alcançado por sua árvore de chamadas, e avaliadas por três LLMs, segundo uma rubrica de seis dimensões, com e sem esse código. As diferenças entre os cenários são estatisticamente significativas nas seis dimensões, mas sua direção depende do avaliador, e, na maioria das ferramentas em que todos os avaliadores mudam a nota, a mudança decorre de informações que o código revela e a descrição omite, como parâmetros não documentados e modos de falha não divulgados.*

Abstract. *The Model Context Protocol (MCP) enables Artificial Intelligence (AI) agents to interact with external systems through tools, whose natural language descriptions guide their usage. Poor-quality descriptions, characterized by the presence of smells, can compromise the effectiveness of these agents. This work investigates whether the quality assessment of these descriptions changes when the evaluator has access to the tool's source code. To this end, 12,166 tools from 206 MCP server repositories on GitHub are extracted, together with the code reached by their call graph, and evaluated by three LLMs against a six-dimension rubric, with and without that code. The differences between scenarios are statistically significant in all six dimensions, but their direction depends on the evaluator, and, for most tools whose score changes across all evaluators, the change stems from information the code reveals and the description omits, such as undocumented parameters and undisclosed failure modes.*

Bacharelado em Engenharia de *Software* - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): João Pedro Batisteli - joaobatisteli@pucminas.br
Orientador acadêmico (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br
Orientador do TCC II: Glender Brás - glendermedeiros@pucminas.br

Belo Horizonte, 28 de junho de 2026.

1. Introdução

O avanço dos modelos de linguagem (*Large Language Models*, LLMs) transformou o paradigma de interação entre humanos e máquinas [Hasan et al. 2026a]. No entanto, a ausência inicial de padrões de comunicação entre LLMs e ferramentas externas gerou uma fragmentação técnica que motivou o surgimento do *Model Context Protocol* (MCP), um protocolo aberto criado para padronizar a integração de modelos de IA com programas e fontes de dados externas [Shi et al. 2026]. A adoção do protocolo tem crescido de forma acelerada, e um levantamento publicado no início de 2026 apontou que seus Kits de Desenvolvimento de Software (*Software Development Kit*, SDK) já registravam aproximadamente 97 milhões de *downloads* mensais [Avaya 2026]. O principal diferencial do MCP reside no uso de ferramentas (*tools*) descritas em linguagem natural, que o agente pode selecionar e invocar em tempo de execução conforme a tarefa a ser resolvida. Esse funcionamento impõe, contudo, uma dependência crítica da qualidade documental, uma vez que o modelo não tem acesso ao código que implementa as ferramentas. As decisões de usá-las, como invocá-las e quais parâmetros fornecer são tomadas com base em suas descrições [Shi et al. 2026, Luo et al. 2025]. Dessa forma, assim como *code smells* comprometem a manutenibilidade de *softwares* tradicionais, descrições ambíguas ou incompletas comprometem o comportamento do agente, podendo induzir erros de planejamento, uso incorreto de parâmetros e alucinações [Hasan et al. 2026b].

Estudos recentes investigam a prevalência de anomalias (ou *smells*) em descrições de ferramentas MCP por meio de análise documental [Hasan et al. 2026b, Guo et al. 2025, Shi et al. 2026]. Essa abordagem avalia se a documentação comunica adequadamente o propósito, os parâmetros e as restrições de uso de uma ferramenta, sem considerar a implementação técnica [Hasan et al. 2026a]. Entretanto, falhas relacionadas à configuração e à documentação representam 38,66% dos problemas observados em servidores MCP [Taraghi et al. 2026]. No MCP, a descrição é um artefato redigido manualmente para um código já existente, e é o código que define o comportamento efetivo da ferramenta; a prevalência de inconsistências entre os dois em servidores reais [Shi et al. 2026] mostra que eles podem divergir. Diante disso, existe uma lacuna na **análise da qualidade das descrições das ferramentas MCP, que, sem o contexto do código, pode não ser suficiente para a avaliação de sua qualidade**.

Essa lacuna é relevante porque inconsistências semânticas podem reduzir a eficácia dos agentes e ampliar a superfície de exposição a vulnerabilidades, incluindo ataques de injeção de comandos [Taraghi et al. 2026], e relatórios de segurança alertam que ambiguidades documentais podem favorecer a execução de ações não autorizadas ou a exposição indevida de dados sensíveis [National Security Agency 2026].

Diante desse problema, este trabalho é orientado pelas seguintes questões de pesquisa: **RQ1**: o fornecimento do código da ferramenta ao avaliador aumenta a precisão da avaliação da qualidade de sua descrição, em relação à avaliação baseada apenas na descrição? **RQ2**: quais atributos da rubrica de qualidade documental são mais afetados pela adição do código da ferramenta? **RQ3**: as diferenças observadas entre os escores dos cenários com e sem código são estatisticamente significativas?

Para respondê-las, este trabalho tem como objetivo geral **investigar se a avaliação da qualidade de descrições de ferramentas MCP é influenciada pela disponibilização do código da ferramenta ao avaliador.** Desse objetivo derivam três objetivos es-

pecíficos, cada um associado a uma questão de pesquisa: (i) Examinar se o fornecimento do código da ferramenta resulta em maior precisão na avaliação da qualidade de sua descrição; (ii) Identificar os atributos de avaliação mais afetados pela adição do código da ferramenta; e (iii) Realizar uma análise estatística sobre os resultados obtidos nas avaliações.

Para isso, as ferramentas de 206 repositórios de servidores MCP de código aberto são avaliadas por três modelos de linguagem atuando como avaliadores (*LLM-as-a-Judge*), que pontuam, em escala Likert de cinco pontos, as seis dimensões da rubrica de qualidade documental de Hasan et al. (2026b), com e sem acesso ao código da ferramenta.

2. Fundamentação Teórica

Esta seção apresenta os conceitos que sustentam a pesquisa. A Seção 2.1 discute a qualidade de software e sua extensão à documentação; a Seção 2.2 descreve o *Model Context Protocol* e o papel que as descrições desempenham na interação entre o agente e as ferramentas; e a Seção 2.3 apresenta o conceito de *smell* aplicado a essas descrições.

2.1. Qualidade de Software

A qualidade, no âmbito da Engenharia de Software, pode ser compreendida como o grau em que um produto de software atende aos requisitos especificados e às necessidades e expectativas de seus usuários e demais partes interessadas [ISO/IEC/IEEE 2017]. Segundo Pressman e Maxim (2016), a qualidade de software envolve tanto a conformidade com requisitos funcionais e de desempenho explicitamente declarados quanto a aderência a padrões de desenvolvimento e a características implícitas esperadas de todo software desenvolvido profissionalmente. Normas internacionais como a ISO/IEC 25010 [ISO/IEC 2011] estabelecem modelos de qualidade que decompõem esse conceito em características mensuráveis, tais como adequação funcional, confiabilidade, usabilidade e manutenibilidade. Essa avaliação não se restringe ao código, estendendo-se também aos artefatos que compõem o software, como documentação, especificações e descrições, cuja clareza e precisão impactam diretamente a compreensão e o uso correto do sistema.

No caso da documentação, a qualidade está associada à capacidade do texto de comunicar corretamente o que o software faz e como deve ser utilizado. Em documentação de APIs, Khan et al. (2021) catalogam problemas recorrentes que dificultam esse uso, como textos inchados ou excessivamente superficiais, informações fragmentadas entre diferentes pontos e descrições que misturam detalhes estruturais com instruções de uso, e mostram, a partir da percepção de profissionais, que esses problemas são frequentes e prejudicam a produtividade. Há ainda uma característica comum a toda documentação: ela é produzida separadamente da implementação, e nada garante que as duas permaneçam alinhadas. Um texto pode, assim, ser claro e bem escrito e, ainda assim, incompleto ou incorreto em relação ao comportamento real do código, divergência que não pode ser detectada quando a avaliação considera apenas o texto. Quando quem consome a documentação é um agente de IA, que decide a partir dela quando e como agir, uma descrição de baixa qualidade deixa de ser uma falha documental isolada e passa a ser um defeito com efeito comportamental observável; é nessa dimensão que este trabalho se concentra.

2.2. Model Context Protocol

O *Model Context Protocol* (MCP) é um padrão aberto que define como aplicações de IA se conectam a fontes de dados e ferramentas externas, de modo análogo ao papel que o USB-C desempenha ao uniformizar a conexão de periféricos de *hardware* [Guo et al. 2025]. Tecnicamente, o protocolo adota uma arquitetura cliente-servidor baseada em mensagens JSON-RPC, na qual um *host* orquestra um ou mais servidores [Shi et al. 2026]. Cada servidor disponibiliza suas capacidades por meio de três primitivas: as ferramentas (*tools*), que executam ações; os recursos (*resources*), que fornecem dados contextuais; e os *prompts*, que oferecem modelos de instrução [Guo et al. 2025]. Esse modelo padronizado dispensa código de integração específico para cada plataforma e favorece a interoperabilidade e o reúso de ferramentas no ecossistema [Hasan et al. 2026a].

Na prática, quando um servidor é conectado, o *host* solicita a lista de ferramentas disponíveis e recebe, para cada uma, um nome, uma descrição em linguagem natural e um esquema JSON dos parâmetros de entrada. Essas informações são inseridas no contexto do modelo, que decide, a partir delas, se alguma ferramenta é útil para a tarefa, qual escolher e com quais argumentos invocá-la; o resultado da execução retorna ao modelo, que segue planejando os próximos passos [Shi et al. 2026]. Em nenhum momento desse ciclo o modelo tem acesso ao código que implementa a ferramenta: a descrição e o esquema são a única representação da ferramenta de que ele dispõe. Informações ausentes da descrição, como restrições de formato, limites de paginação ou condições de erro, só podem ser descobertas pelo agente durante a execução, por tentativa e erro, o que contribui para as falhas no uso de interfaces de ferramentas observadas em *benchmarks* de agentes [Luo et al. 2025].

2.3. Descrições de Ferramentas e *Smells*

No MCP, as ferramentas dependem de descrições em linguagem natural como principal guia para os agentes. Diferentemente de sistemas tradicionais, que utilizam assinaturas de código estritas, a orquestração por IA exige que o propósito, os parâmetros e as restrições de cada funcionalidade sejam explicados textualmente [Hasan et al. 2026b]. Uma descrição ambígua ou incompleta pode induzir o modelo a erros de planejamento, resultando em chamadas inválidas ou no uso de parâmetros inadequados [Shi et al. 2026]; a descrição funciona, assim, como um artefato de especificação.

O conceito de *smell* refere-se a características estruturais que sugerem problemas de qualidade em um artefato, ainda que não configurem defeitos funcionais [Hasan et al. 2026b]. Embora originalmente associado ao código, o conceito foi estendido a artefatos em linguagem natural, como os *requirements smells*, que identificam ambiguidades em especificações de requisitos [Femmer et al. 2017], e os *documentation smells*, que apontam problemas de qualidade em documentação de APIs [Khan et al. 2021]. No âmbito do MCP, o termo aplica-se a ambiguidades, inconsistências e omissões nas descrições de ferramentas, como a ausência de especificação dos parâmetros, do comportamento esperado e das limitações da ferramenta [Hasan et al. 2026b]. Por exemplo, uma ferramenta cuja descrição se limita a "processa os dados" configura um *smell* de vagueza: o texto não especifica quais dados são aceitos, qual transformação é aplicada nem qual saída é produzida.

3. Trabalhos Relacionados

Esta seção apresenta estudos sobre a qualidade das descrições MCP, o desempenho de agentes no ecossistema e o uso de LLMs como avaliadores, que fundamentam a lacuna desta pesquisa.

Hasan et al. (2026b) investigaram a qualidade das documentações de 856 ferramentas no ecossistema MCP por meio de uma análise empírica fundamentada em LLMs. O estudo aplicou uma rubrica de seis dimensões focada na clareza e completude do conteúdo textual. Os resultados revelaram que 97,1% das descrições possuem ao menos um *smell* e que complementar as descrições aumentou em 5,85 pontos percentuais a taxa de sucesso de agentes, à custa de 67% mais passos de execução. Em uma versão posterior, os autores também fornecem ao juiz a implementação da ferramenta, em 200 ferramentas do MCP-Universe e com um único modelo, e observam aumento sobretudo em *Parameter Explanation*. O presente estudo parte dessa investigação e a amplia em três direções: a escala, com 12.166 ferramentas de 206 repositórios; o uso de três juízes de provedores distintos, que permite verificar se o efeito do código depende do avaliador; e a caracterização qualitativa do tipo de informação que o código revela e a descrição omite.

Shi et al. (2026) investigam a prevalência e os impactos de inconsistências semânticas entre descrições de ferramentas MCP e seus respectivos códigos, propondo uma taxonomia de inconsistências e um detector baseado em LLM aplicado em larga escala a servidores reais. Os resultados revelaram que 9,93% das ferramentas analisadas apresentam inconsistências e que 35% dos servidores possuem ao menos um caso, com riscos de seleção incorreta de ferramentas, efeitos colaterais não declarados e falhas de segurança. Esse estudo sustenta a premissa desta pesquisa ao demonstrar que informações relevantes sobre o comportamento de uma ferramenta frequentemente não estão presentes em sua descrição.

Guo et al. (2025) mapeiam a adoção do MCP com um rastreador que analisa mais de 8.400 projetos e constatam que mais da metade dos projetos listados são inválidos, abandonados ou de baixo valor agregado. O estudo caracteriza os repositórios, mas não examina a consistência entre descrições e implementação; o dado de baixa maturidade também fundamenta o critério de popularidade adotado na seleção da amostra (Seção 4).

Luo et al. (2025) propuseram o MCP-Universe, um *benchmark* que avalia modelos de linguagem em servidores MCP reais por meio de avaliadores baseados em execução. Ao avaliar mais de 20 modelos em 231 tarefas distribuídas por 11 servidores, os resultados revelaram que mesmo modelos de fronteira atingem taxas de sucesso abaixo de 45%, com falhas frequentemente associadas ao uso incorreto de interfaces de ferramentas pouco familiares. O presente trabalho não mede o desempenho de agentes; ele examina um fator anterior a essas falhas, a qualidade da informação que a descrição oferece ao agente sobre a interface da ferramenta, e se o código revela omissões nessa informação.

Zheng et al. (2023) investigam a viabilidade da abordagem *LLM-as-a-Judge*, comparando julgamentos do GPT-4 com avaliações humanas em *benchmarks* de conversação, e mostram concordância superior a 80%, comparável à observada entre os próprios especialistas. Esse resultado fundamenta o uso de avaliadores baseados em LLM neste estudo.

Em síntese, esses estudos mostram que a qualidade das descrições MCP é um fator crítico e que texto e implementação podem divergir; a incorporação do código à avaliação, porém, foi examinada apenas em pequena escala e com um único avaliador. Este trabalho investiga como essa incorporação influencia a avaliação em larga escala e com múltiplos juízes.

4. Materiais e Métodos

A pesquisa caracteriza-se como um estudo empírico, quantitativo e comparativo, cujo fluxo está resumido na Figura 1. O processo inicia-se com a coleta de repositórios MCP públicos, seguida pela extração das ferramentas e do código que as implementa. Em seguida, a qualidade das descrições é avaliada em dois cenários: no primeiro, apenas com a descrição em linguagem natural; no segundo, com a mesma descrição acompanhada pelo código correspondente. As notas são então consolidadas em uma base pareada, da qual se deriva a amostra de divergência confirmada, e, por fim, submetidas a uma análise comparativa, estatística e qualitativa. O detalhamento de cada etapa será discutido nas próximas seções: a Etapa 1 na Seção 4.1, a Etapa 2 na Seção 4.2, as Etapas 3 e 4 na Seção 4.3 e a Etapa 5 na Seção 4.4.

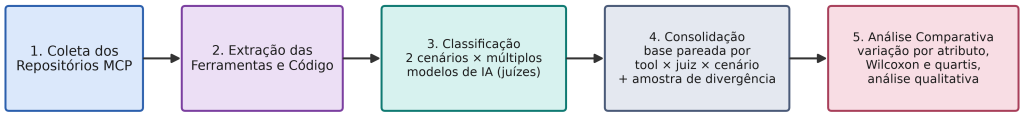


Figura 1. Fluxo metodológico adotado na pesquisa

4.1. Coleta dos Repositórios MCP

Foram minerados repositórios públicos de servidores MCP no GitHub. Neste trabalho, *repositório* designa o projeto público no GitHub, que é a unidade de coleta e de amostragem, e *servidor* designa o servidor MCP implementado nele. Como um mesmo repositório pode reunir mais de um servidor, como nos repositórios que agrupam vários servidores de exemplo, todas as contagens da coleta são feitas por repositório. Como a API REST de busca de código limita cada consulta a 1.000 resultados [GitHub 2026], a busca foi segmentada em uma subconsulta por combinação entre um pacote do SDK MCP (por exemplo, `@modelcontextprotocol/sdk`) e um arquivo de manifesto em que ele pode ser declarado (por exemplo, `package.json`). Metadados ausentes na busca de código (estrelas, condição de *fork* e linguagem principal) foram obtidos pela API GraphQL, e os resultados foram deduplicados por identificador. A busca pela dependência do SDK, e não por palavras-chave no nome ou na descrição do repositório, aumenta a chance de que os candidatos implementem de fato um servidor MCP, em vez de apenas mencionarem o protocolo, embora o mesmo pacote também seja usado por clientes MCP.

Dos 12.058 repositórios candidatos, isto é, repositórios distintos que declaram uma dependência do SDK MCP, 723 (6,0%) atenderam aos critérios de elegibilidade: não ser *fork*, estar em uma linguagem-alvo e possuir ao menos 100 estrelas. O número de estrelas é usado na mineração de repositórios como medida de popularidade [Borges et al. 2016], e o limiar de 100 estrelas segue Han et al. (2019), que o definiram a partir de um levantamento com desenvolvedores do GitHub como fronteira entre projetos

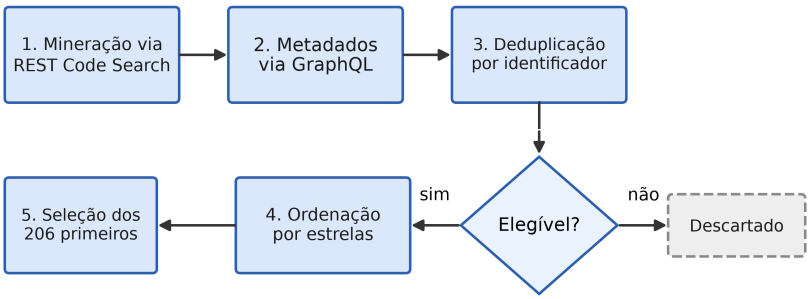


Figura 2. Funcionamento detalhado da Etapa 1 (Coleta dos Repositórios)

populares e não populares; ele exclui projetos experimentais ou de baixo valor agregado, os *toy repositories*. Os elegíveis foram ordenados por estrelas, priorizando os repositórios mais adotados pela comunidade, nos quais deficiências de descrição alcançam um número maior de usuários e agentes, e selecionados até completar 206 repositórios, o dobro da amostra de Hasan et al. (2026b); um candidato só permanecia na amostra se a Etapa 2 localizasse nele ao menos uma ferramenta, sendo substituído pelo próximo da lista caso contrário (laço de *backfill*). A Figura 2 detalha o funcionamento dessa etapa.

4.2. Extração das Ferramentas e do Código Relacionado

Cada repositório selecionado foi clonado localmente por clonagem rasa (*shallow clone*), que copia apenas a versão mais recente do código, sem o histórico de *commits*, desnecessário para a análise, e analisado por um algoritmo de mineração baseado em análise estática (*tree-sitter*), que localiza as ferramentas declaradas segundo o padrão do SDK MCP de cada linguagem e extrai sua descrição em linguagem natural e sua localização no código, conforme detalhado na Figura 3. Se nenhuma ferramenta é encontrada no repositório, condição representada pelo losango da figura, ele é desqualificado e substituído pelo próximo candidato da lista, no laço de *backfill* descrito na Seção 4.1.

Como os servidores MCP possuem estruturas variadas, extrair apenas o método raiz de uma ferramenta pode não ser suficiente para capturar sua lógica. Para isso, foi construída uma árvore de chamadas (*call graph*) a partir do método inicial. O algoritmo tem três fases, executadas uma vez por repositório. Na primeira, o repositório é indexado: registram-se todas as funções e métodos nele definidos e, para cada arquivo, os módulos que ele importa. Na segunda, a árvore de cada ferramenta é construída a partir do seu método, expandida recursivamente até três níveis de profundidade: o próprio método constitui o primeiro nível, os métodos chamados diretamente por ele, quando resolvidos, formam o segundo, e as chamadas efetuadas a partir destes compõem o terceiro. A resolução das chamadas é heurística e baseada em nome: cada chamada é associada, nesta ordem, a um método da própria classe, a uma função do mesmo arquivo, a uma função de um módulo importado ou a uma definição de nome único no repositório; quando há mais de um candidato, escolhe-se o mais próximo na estrutura de diretórios e o nó é sinalizado como ambíguo, e chamadas não resolvidas, como as de bibliotecas externas, tornam-se folhas, sem recursão. O limite de três níveis contém o custo da análise e do texto enviado ao avaliador. Na terceira fase, esse texto é composto: a árvore é percorrida a partir da raiz e, para cada nó resolvido, o trecho de código correspondente é copiado do repositório,

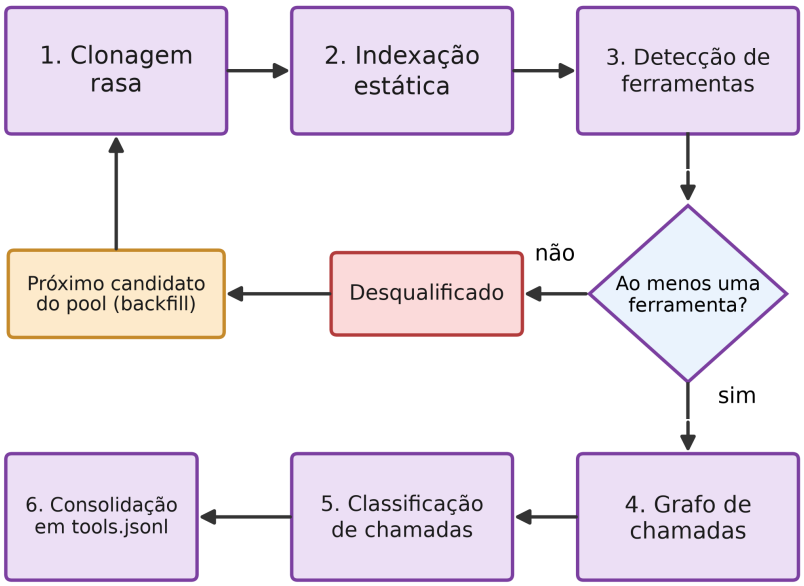


Figura 3. Funcionamento detalhado da Etapa 2 (Extração das Ferramentas)

com a indicação do arquivo e das linhas de origem; o resultado é limitado a 32.000 caracteres. A Figura 4 ilustra o processo: o método da ferramenta (nível 1) chama um serviço (nível 2), que por sua vez chama outro serviço e um repositório de dados (nível 3); os trechos desses quatro métodos são enviados ao avaliador, e as chamadas feitas a partir do nível 3 (nível 4) não são incluídas.

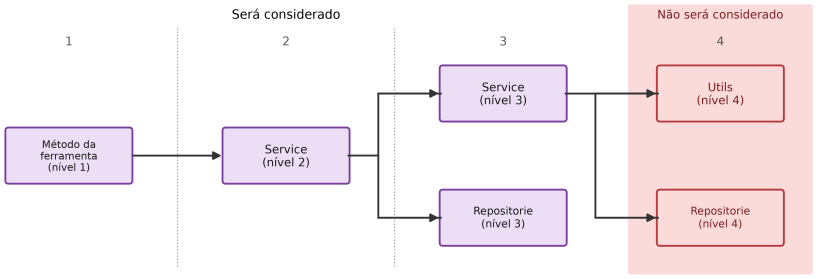


Figura 4. Exemplo de extração da árvore de chamadas de uma ferramenta

A Etapa 2 extraiu 12.166 ferramentas dos 206 repositórios, distribuídos em nove linguagens, uma média de 59,1 ferramentas por repositório. Python (79 repositórios e 7.246 ferramentas) e TypeScript (45 repositórios e 3.162 ferramentas) concentram 60% dos repositórios e 85,5% das ferramentas, e a distribuição por repositório é fortemente assimétrica: os três repositórios com mais ferramentas respondem por 46,7% do conjunto (20,4%, 18,5% e 7,8%). Como as análises tratam cada ferramenta como uma observação, esses três repositórios pesam quase metade em qualquer média calculada sobre o conjunto, e um padrão de escrita próprio deles pode parecer geral. Por isso, as proporções e médias reportadas descrevem o conjunto de ferramentas, e não o repositório típico.

4.3. Avaliação das Ferramentas e Consolidação dos Resultados

Cada ferramenta foi avaliada em dois cenários: (i) sem código, recebendo nome, repositório de origem e descrição, os mesmos campos usados no *script* de avaliação de Hasan et al. (2026b); e (ii) com código, recebendo adicionalmente o código-fonte da ferramenta, isto é, os trechos de código dos métodos que compõem sua árvore de chamadas (Seção 4.2), e não a árvore em si. A descrição é a mesma nos dois cenários; a única diferença entre eles é a presença do código. O esquema de entrada (*inputSchema*) não é fornecido ao juiz, de modo a isolar o texto da descrição, que é o objeto avaliado. A especificação do MCP trata descrição e esquema como campos distintos da ferramenta [Model Context Protocol 2026], e a documentação da Anthropic aponta a descrição como o fator mais importante para o desempenho no uso de ferramentas, recomendando que ela própria explique o significado de cada parâmetro [Anthropic 2026]. Faghih et al. (2025) adotam o mesmo recorte ao alterar apenas a descrição das ferramentas, o que basta para mudar a frequência com que os modelos as escolhem. A Figura 5 detalha o funcionamento dessa etapa.

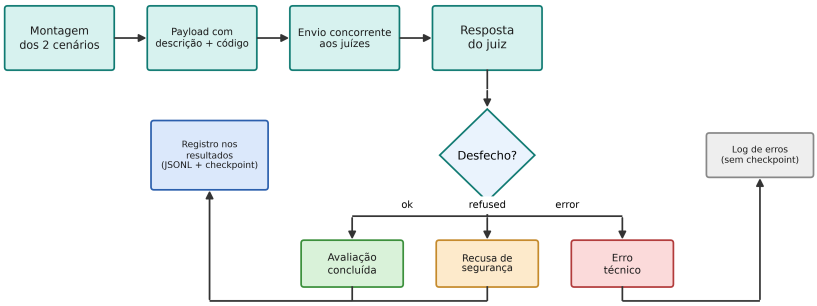


Figura 5. Funcionamento detalhado da Etapa 3 (Classificação das Ferramentas)

Em ambos os cenários, modelos de linguagem atuam como juízes (*LLM-as-a-Judge*) e pontuam, segundo as definições dos seis componentes de Hasan et al. (2026b) e uma escala Likert única de cinco pontos: *Purpose* (clareza da finalidade da ferramenta), *Guidelines* (orientações sobre quando e como utilizá-la), *Limitations* (restrições, premissas e casos de falha), *Parameter Explanation* (documentação dos parâmetros de entrada), *Length and Completeness* (detalhamento e cobertura) e *Examples* (presença de exemplos de uso).

Foram utilizados três juízes de proveniências e custos distintos: DeepSeek-V4.1-Flash e Gemini 3.5 Flash-Lite, acessados via API, e Qwen3-14B, executado localmente via Ollama. A combinação reduz o risco de que um padrão observado seja específico de um único provedor. Não se pressupõe que os juízes concordem entre si: as diferenças entre eles não são descartadas como ruído, mas analisadas como parte dos resultados. As notas foram consolidadas em uma base pareada por ferramenta, juiz e cenário. Dela deriva-se a **amostra de divergência confirmada**: as ferramentas em que, para ao menos um componente da rubrica, todos os juízes atribuem notas diferentes entre os cenários sem e com código. Para cada uma, reúnem-se a justificativa (*reasoning*) de cada juiz nos dois cenários e o código correspondente. O *prompt* dos juízes, idêntico nos dois cenários, está no Apêndice A.

4.4. Análise Comparativa dos Experimentos

A análise comparativa combina três abordagens complementares, cada uma voltada a uma questão de pesquisa: uma análise descritiva por juiz e atributo (RQ2), testes estatísticos sobre as notas pareadas (RQ3) e uma análise qualitativa das divergências (RQ1). Na análise descritiva, para cada juiz e componente, calculam-se a proporção de ferramentas cuja nota diminuiu, se manteve ou aumentou com a inclusão do código e a distribuição das notas nos dois cenários, identificando os atributos mais afetados, entendidos como os de maior proporção de ferramentas com nota alterada (objetivo específico ii). Os resultados são apresentados separadamente por juiz, sem agregação, para que diferenças de comportamento entre os avaliadores não sejam mascaradas.

Para verificar a significância das diferenças (objetivo específico iii), as notas dos três juízes são agregadas pela média, por ferramenta e componente, e aplica-se a cada componente o Teste de Postos Sinalizados de Wilcoxon bilateral [Wilcoxon 1945], adequado a amostras pareadas sem suposição de normalidade, com nível de significância de 5% ($\alpha = 0{,}05$). Não se aplica correção para comparações múltiplas porque são apenas seis testes, um por atributo, e uma correção de Bonferroni exigiria $p < 0{,}0083$; a Seção 5.3 mostra que todos os resultados atendem também a esse limiar; pares com diferença nula são descartados, conforme o tratamento padrão de empates do teste, e o número de pares com diferença (n) é reportado. Complementarmente, aplica-se uma análise de transição de quartil: os quartis da distribuição da nota média no cenário sem código definem quatro faixas por componente, e cada ferramenta é reclassificada segundo a faixa em que sua nota média com código se enquadra, reportando-se a proporção que mantém, sobe ou desce de faixa. A transição de quartil complementa o teste porque, com milhares de pares, até diferenças pequenas tendem a ser significativas: ela indica quantas ferramentas mudam de patamar de qualidade, e não apenas se a diferença existe.

Para responder à RQ1, é preciso saber o que, no código, levou os juízes a mudar a nota. Para isso, conduz-se uma análise qualitativa sobre a amostra de divergência confirmada, em três etapas (Figura 6; *prompts* no Apêndice B). Na primeira, que corresponde às fases de familiarização e codificação inicial de uma análise temática [Braun and Clarke 2006], um modelo de linguagem (Gemini) lê, para cada ferramenta, as justificativas dos juízes e o código, e escreve uma justificativa única sobre o que o código revelou e a descrição omitia. Quando as justificativas não apontam nenhum elemento concreto do código, o modelo é instruído a declarar isso explicitamente, e não a deduzir uma causa.

Na segunda etapa, que corresponde às fases de busca, revisão e definição de temas, um segundo modelo (DeepSeek) recebe todas as justificativas de uma vez e propõe, a partir delas, categorias de mecanismo, cada uma sustentada por mais de uma ferramenta. Essa etapa foi executada duas vezes, cada uma produzindo 13 categorias, e as duas listas foram unidas pelo mesmo modelo em 16 categorias. Na terceira etapa, uma análise de conteúdo dirigida [Hsieh and Shannon 2005] atribui a cada justificativa zero ou mais dessas categorias, uma ferramenta por vez, para evitar a perda de desempenho dos modelos em listas longas [Liu et al. 2024].

Por fim, a classificação é verificada manualmente em 100 ferramentas. Pela Fórmula de Cochran [Chaokromthong et al. 2021], 96 ferramentas bastam para estimar, com 95% de confiança e margem de erro de 10%, a proporção de classificações corre-

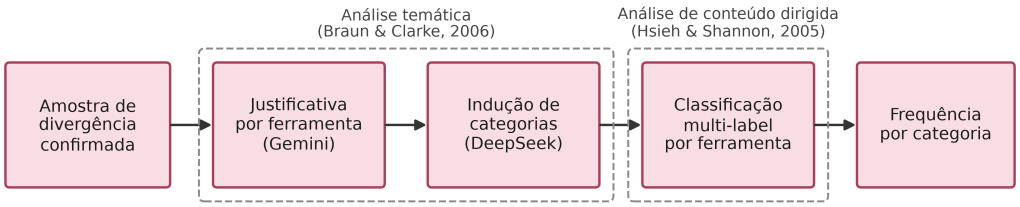


Figura 6. Fluxo detalhado da análise qualitativa sobre a amostra de divergência confirmada

tas em toda a amostra de divergência confirmada, considerando a proporção mais desfavorável (0,5); o valor foi arredondado para 100, como em Hasan et al. (2026b). A amostra é estratificada com vagas distribuídas igualmente entre as categorias (mais `none`), priorizando repositórios e componentes ainda não representados. Cada um dos dois avaliadores lê, sem consultar o outro, a justificativa sintetizada e o *reasoning* original dos juízes da ferramenta e indica se concorda com as categorias atribuídas pelo modelo.

5. Resultados

Esta seção apresenta os resultados da avaliação por rubrica descrita na Seção 4, com base nas avaliações dos três juízes, DeepSeek-V4.1-Flash, Gemini 3.5 Flash-Lite e Qwen3-14B, que avaliaram as mesmas 12.166 ferramentas nos dois cenários experimentais (sem código e com código).

5.1. RQ1: O fornecimento do código da ferramenta ao avaliador aumenta a precisão da avaliação da qualidade de sua descrição, em relação à avaliação baseada apenas na descrição?

A RQ1 investiga se o acesso ao código torna mais precisa a avaliação da descrição. Antes de analisar os resultados, cabe observar que o aumento ou a diminuição da nota não indica, por si só, uma melhora ou piora na precisão da avaliação: a divergência pode refletir tanto um ganho real de precisão quanto viés ou ruído do avaliador. Por isso, esta seção não se limita à direção da mudança e busca identificar o **mecanismo** pelo qual o código altera a nota, isto é, que tipo de informação presente no código e ausente da descrição levou os juízes a reavaliá-la.

A análise qualitativa (Seção 4.4) foi conduzida sobre as 2.273 ferramentas da amostra de divergência confirmada (18,7% das 12.166 avaliadas). A análise temática indutiva produziu 16 categorias de mecanismo, cada uma descrevendo um tipo de informação que o código revela e que a descrição não comunica, como pode ser visto na Tabela 1. A tabela informa, para cada categoria, em quantas ferramentas da amostra ela foi atribuída e a proporção correspondente; como uma ferramenta pode receber mais de uma categoria, a soma das proporções excede 100%.

Superfície de Entrada Não Documentada domina a amostra (57,4%): o achado mais comum é que o código aceita parâmetros, filtros ou opções que a descrição não menciona. Modos de Falha e Erros Não Divulgados (26,1%) e Estrutura Concreta da Saída (19,5%) seguem como segundo e terceiro mecanismos mais recorrentes, e mesmo a categoria menos frequente, Efeitos Colaterais Não Divulgados, aparece em 56 ferramentas. A classificação deixou apenas 4,1% das ferramentas (94 de 2.273) sem nenhuma

Tabela 1. Frequência de cada categoria de mecanismo na amostra de divergência confirmada (n = 2.273; multi-*label*, a soma excede 100%)

Categoria	Ocorrências	%
Superfície de Entrada Não Documentada	1.305	57,4%
Modos de Falha e Erros Não Divulgados	593	26,1%
Estrutura Concreta da Saída	443	19,5%
Regras de Validação e Contratos de Formato Não Divulgados	435	19,1%
Limites Operacionais, Padrões e Paginação Não Divulgados	386	17,0%
Contexto de Fluxo e Ativação	292	12,8%
Escopo de Processamento e Mecânica Interna Subestimados	278	12,2%
Confirmação de Encapsulamento Trivial	217	9,5%
Valores Fixos e Padrões Divergentes	183	8,1%
Dependências e Pré-requisitos de Ambiente Ocultos	165	7,3%
Semântica de Correspondência e Normalização de Entrada	109	4,8%
Controles de Acesso, Papel e Autorização	104	4,6%
Parâmetros Fantasmas ou Divergentes	79	3,5%
Funcionalidade Anunciada Não Implementada	73	3,2%
Sistema-Alvo Divergente	58	2,6%
Efeitos Colaterais Não Divulgados	56	2,5%
(nenhuma categoria)	94	4,1%

categoria (`none`), o que indica que o conjunto de categorias cobre a grande maioria dos casos. A definição detalhada de cada categoria pode ser vista no Apêndice C.

Na verificação manual, as categorizações atribuídas pelo modelo às 100 ferramentas da amostra foram avaliadas individualmente por dois avaliadores, que marcaram cada uma como "concordo" ou "discordo". O avaliador A concordou com 86% das categorizações do modelo e o avaliador B, com 80%, e houve divergência entre os dois em 30% das respostas. A maior parte das categorizações foi, portanto, aceita pelos avaliadores, mas a divergência entre eles indica que a fronteira entre algumas categorias é sujeita a interpretação.

Em resposta à RQ1, os resultados não permitem afirmar que o código torna a avaliação mais precisa: como discutido no início desta seção, isso exigiria comparar as notas com um julgamento humano de referência, inexistente neste estudo (Seção 6). O que eles mostram é que a mudança de nota não é aleatória. Em 89,3% das ferramentas da amostra (2.029 de 2.273), ela tem uma causa concreta no código, isto é, uma informação que a descrição omite ou contradiz e que um avaliador sem acesso ao código não teria como verificar; nos 10,7% restantes (sem categoria ou apenas Confirmação de Encapsulamento Trivial), o código não revela informação nova, e a mudança reflete apenas uma releitura do mesmo texto. O código, portanto, dá ao avaliador acesso a deficiências que a leitura da descrição sozinha não revela, mas se a nota resultante é mais correta depende de como cada juiz usa essa informação, o que a RQ2 examina.

5.2. RQ2: Quais atributos da rubrica de qualidade documental são mais afetados pela adição do código da ferramenta?

A Figura 7 apresenta, para cada juiz, a proporção de casos em que a avaliação com código resultou em diminuição, manutenção ou aumento da nota de cada atributo, e a Figura 8 compara a distribuição das notas nos dois cenários.

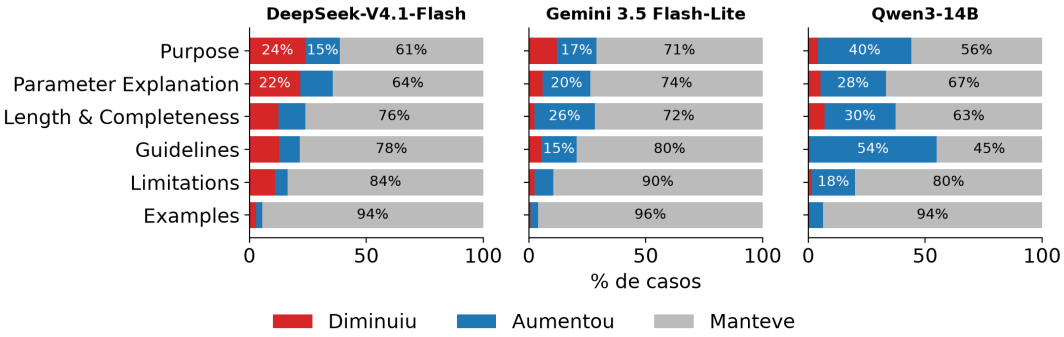


Figura 7. Distribuição proporcional da alteração nas pontuações por atributo e por juiz.

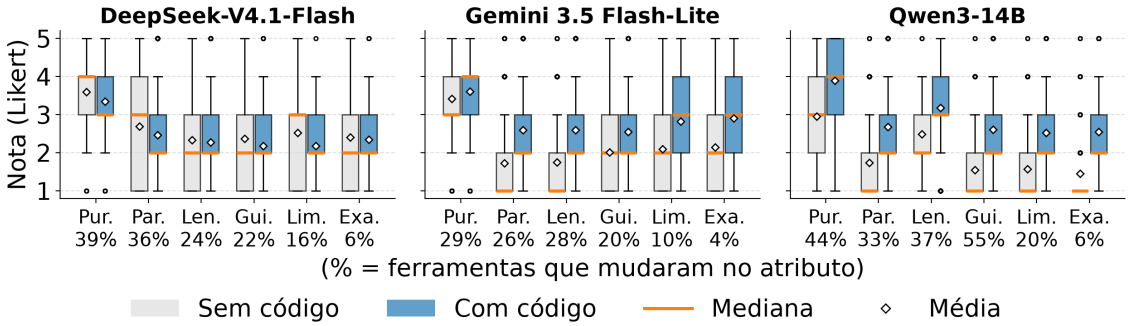


Figura 8. Notas com e sem código, por juiz, apenas nos casos com alteração de pontuação. Pur.: *Purpose*; Par.: *Parameter Explanation*; Len.: *Length & Completeness*; Gui.: *Guidelines*; Lim.: *Limitations*; Exa.: *Examples*.

Três padrões se destacam. O primeiro é que a direção da mudança depende do juiz. O DeepSeek reduz a nota média nos seis atributos, de $-0{,}003$ em *Examples* a $-0{,}099$ em *Purpose*, e, em todos eles, suas notas descem com mais frequência do que sobem; em *Purpose*, por exemplo, 24,0% das notas descem e 14,6% sobem. O Gemini e o Qwen3 fazem o oposto e elevam a nota média nos seis atributos: o Gemini, de $+0{,}030$ (*Examples*) a $+0{,}240$ (*Length & Completeness*), e o Qwen3, com mudanças maiores, de $+0{,}070$ (*Examples*) a $+0{,}581$ (*Guidelines*), atributo em que 54,5% das suas notas sobem e apenas 0,4% descem. A Figura 8 mostra o mesmo padrão nas distribuições: nos casos em que a nota muda, a mediana com código é menor no DeepSeek em *Purpose*, *Parameter Explanation* e *Limitations*, e maior no Gemini e no Qwen3 em quase todos os atributos.

O segundo padrão diz respeito aos atributos. *Purpose* é o que mais muda no DeepSeek e no Gemini, com nota alterada em 38,6% e 28,9% das ferramentas, e o segundo que mais muda no Qwen3 (44,1%), atrás de *Guidelines* (54,9%). *Limitations* e *Examples* são os que menos mudam nos três juízes: *Limitations*, entre 10,3% e 20,1% das ferramentas, e *Examples*, entre 4,0% e 6,4%, o que é esperado, pois o código raramente contém exemplos de uso. O terceiro padrão é a baixa concordância entre os juízes: entre as ferramentas em que os três alteram a nota de um mesmo atributo, os três a alteram no mesmo sentido em apenas 25,2% dos casos em *Limitations* e, no máximo, em 49,6% dos casos em *Parameter Explanation*. Em mais da metade das vezes, portanto, o mesmo código leva um juiz a subir a nota e outro a baixá-la.

Para entender por que os juízes divergem, a direção das mudanças foi cruzada com as categorias de mecanismo da RQ1, na amostra de divergência confirmada. Nas categorias em que o código revela uma omissão da descrição, a reação coerente com a rubrica seria reduzir a nota, e é o que faz o DeepSeek: em Superfície de Entrada Não Documentada, apenas 39,8% das suas notas alteradas sobem, e em Dependências e Pré-requisitos de Ambiente Ocultos, 28,1%. O Gemini e o Qwen3, ao contrário, elevam a nota nessas mesmas ferramentas em 81,7% e 93,1% dos casos na primeira categoria e em 69,6% e 94,5% na segunda. O contraste persiste até em Parâmetros Fantasmas ou Divergentes, em que a descrição documenta parâmetros que o código não possui: o DeepSeek eleva a nota em 20,9% das mudanças, o Gemini em 51,4% e o Qwen3 em 87,2%. Já em Confirmação de Encapsulamento Trivial, em que o código apenas confirma o que a descrição afirma, o DeepSeek também tende a elevar a nota (62,9% das mudanças), sua maior proporção entre as categorias. O DeepSeek, portanto, reage ao conteúdo do código, elevando a nota quando ele confirma a descrição e reduzindo quando revela omissões, enquanto o Gemini e o Qwen3 elevam a nota quase independentemente do que o código revela.

As justificativas dos juízes indicam a causa dessa diferença. Na ferramenta `get_account_balance`, de um servidor bancário, a descrição não menciona o parâmetro `account`, que só aparece no esquema de entrada contido no código. Com o código, os dois juízes citam o mesmo fato e chegam a conclusões opostas em *Parameter Explanation*: o DeepSeek reduz a nota de 3 para 1, porque o parâmetro "só é descrito no *inputSchema*, que é separado da descrição em avaliação", e o Gemini eleva a nota de 1 para 3, porque o parâmetro "está presente no esquema com uma breve descrição". A diferença entre os juízes não está, portanto, no que encontram no código, mas no que consideram parte da descrição: o Gemini e o Qwen3 tendem a creditar à descrição o que aprenderam pelo código, avaliando a ferramenta como um todo, enquanto o DeepSeek mantém a separação entre o texto avaliado e o código usado para verificá-lo. Em resposta à RQ2, os atributos mais afetados são *Purpose*, *Parameter Explanation*, *Length & Completeness* e *Guidelines*, e os menos afetados, *Limitations* e *Examples*; a direção do efeito, contudo, depende de como cada juiz interpreta o papel do código na avaliação.

5.3. RQ3: As diferenças observadas entre os escores dos cenários com e sem código são estatisticamente significativas?

A Tabela 2 reúne, por atributo, o resultado do Teste de Wilcoxon e da transição de quartil, ambos calculados sobre a média das notas dos três juízes (Seção 4.4).

Os seis atributos apresentam diferenças estatisticamente significativas entre os cenários ($p < 0{,}05$). Como o número de pares é elevado, até diferenças pequenas tendem a ser significativas, e a transição de quartil ajuda a dimensioná-las: *Purpose*, *Parameter Explanation*, *Length & Completeness* e *Guidelines* mudam de faixa em 31,8%, 34,3%, 34,5% e 39,0% das ferramentas, respectivamente, enquanto *Limitations* e *Examples* mudam em apenas 15,8% e 4,5%. Esses dois atributos são também os que têm menos pares com nota alterada (n), e, em todos os atributos, subir de faixa é mais frequente do que descer.

O Teste de Wilcoxon indica que as distribuições diferem, mas não em que sentido. A transição de quartil mostra que subir de faixa supera descer em todos os atributos, de cerca de 2 para 1 em *Purpose* (21,6% contra 10,2%) a mais de 25 para 1 em *Guidelines* (37,6% contra 1,4%): na média dos juízes, o código tende a elevar a nota. Esse

Tabela 2. Teste de Postos Sinalizados de Wilcoxon e transição de quartil das notas com e sem código, por atributo (média dos três juízes; pareado por ferramenta). n: pares com diferença não nula; faixas de quartil definidas pela distribuição sem código.

	Wilcoxon			Quartil		
Atributo	n	W	p-valor	Manteve	Subiu	Desceu
Purpose	7671	8572745,5	$4{,}44 \times 10^{-224}$	68,2%	21,6%	10,2%
Parameter Explanation	7098	7708245,5	$8{,}09 \times 10^{-178}$	65,7%	26,3%	8,1%
Length & Completeness	6814	3786993,0	$< 10^{-300}$	65,5%	28,5%	6,0%
Guidelines	7635	3720475,0	$< 10^{-300}$	61,0%	37,6%	1,4%
Limitations	3365	1385458,5	$8{,}40 \times 10^{-148}$	84,2%	13,1%	2,7%
Examples	1261	151287,0	$1{,}44 \times 10^{-82}$	95,5%	3,9%	0,6%

resultado deve ser lido em conjunto com a RQ2, pois a média reflete o predomínio de Gemini e Qwen3, que elevam as notas, sobre o DeepSeek, que as reduz, e não um consenso. *Purpose* ilustra esse efeito: é o atributo com mais pares alterados ($n = 7.671$) e mais quedas de faixa (10,2%), o que é coerente com mudanças em sentidos opostos que se compensam parcialmente na média. *Guidelines*, o atributo que mais muda de faixa (39,0%), quase sempre para cima, sugere que o avaliador, ao compreender pelo código o fluxo de uso, passe a considerar a descrição mais orientativa do que ela de fato é, hipótese retomada na Seção 5.4. Já *Examples* mantém a faixa em 95,5% das ferramentas, pois o código raramente contém exemplos de uso.

Em termos práticos, em quatro atributos cerca de uma em cada três ferramentas muda de faixa quando o avaliador tem acesso ao código, proporção suficiente para alterar quais descrições seriam consideradas adequadas ou deficientes caso a avaliação fosse usada para priorizar a documentação ou filtrar servidores. Em resposta à RQ3, as diferenças são estatisticamente significativas nos seis atributos e, em quatro deles, grandes o bastante para alterar a classificação de cerca de um terço das ferramentas.

5.4. Discussão

Analisados em conjunto, os resultados indicam que a qualidade de uma descrição MCP não pode ser plenamente avaliada apenas pelo texto, o que tem consequência direta para o agente, que escolhe e invoca a ferramenta somente a partir da descrição e do esquema de parâmetros (Seção 2.2). As categorias mais frequentes na RQ1 correspondem às informações de que ele precisa em cada passo: Superfície de Entrada Não Documentada (57,4%) pode afetar a escolha dos argumentos, embora parte desses parâmetros possa constar do esquema de entrada; Modos de Falha e Erros Não Divulgados (26,1%) dificultam a recuperação de erros; e Estrutura Concreta da Saída (19,5%) e Limites Operacionais, Padrões e Paginação Não Divulgados (17,0%) afetam a interpretação do resultado, como perceber que ele está paginado ou incompleto. São falhas desse tipo que *benchmarks* de agentes associam ao uso incorreto de ferramentas [Luo et al. 2025] e que Hasan et al. (2026b) relacionam a *smells* nas descrições. A qualidade da descrição, portanto, não é apenas um atributo documental: ela pode condicionar o comportamento do agente, embora Hasan et al. (2026b) não tenham observado ganho de desempenho ao reescrever as descrições com base no código, o que indica que esse efeito depende da tarefa e da ferramenta.

Há, contudo, uma tensão entre os resultados. Se o código revela sobretudo omissões, seria esperado que um avaliador mais bem informado reduzisse as notas, como faz o DeepSeek; Gemini e Qwen3, entretanto, tendem a elevá-las. Duas explicações são compatíveis com esse padrão. Na primeira, o código confirma que a descrição está correta naquilo que afirma, como na categoria Confirmação de Encapsulamento Trivial (9,5%), e o avaliador passa a julgá-la com mais confiança. Na segunda, o avaliador atribui à descrição informações que só obteve pelo código, avaliando a ferramenta como um todo, e não o texto que o agente efetivamente recebe; nesse caso, uma nota mais alta não indicaria uma descrição melhor para o agente, mas apenas um avaliador mais bem informado do que ele. A análise por categoria da RQ2 favorece a segunda explicação: o Gemini e o Qwen3 elevam a nota mesmo quando o código revela omissões ou parâmetros inexistentes, situações em que não há o que confirmar, e, como no exemplo discutido na RQ2, suas justificativas tratam o que está no código como parte da descrição.

A divergência entre os juízes reforça esse ponto: diante da mesma descrição e do mesmo código, três modelos chegam a conclusões opostas, mesmo guiados por uma rubrica explícita. Um aumento de nota com o código, observado com um único juiz, não pode ser lido diretamente como ganho de confiabilidade da avaliação, pois a direção do efeito depende do avaliador e de como ele interpreta o papel do código. Na prática, as categorias da RQ1 funcionam como uma lista de verificação do que costuma faltar nas descrições, e ferramentas de avaliação de descrições se beneficiariam do código desde que o usassem para apontar o que o texto omite, e não como contexto que o avaliador possa confundir com a própria descrição.

6. Ameaças à Validade

Validade de construto. Não há, neste estudo, um padrão-ouro humano de qualidade das descrições: a precisão da avaliação é inferida a partir do mecanismo pelo qual a nota muda (RQ1), não medida contra um julgamento de referência. Assim, demonstra-se que a mudança de nota tem causa identificável no código, mas não que a nota resultante seja, em sentido absoluto, mais correta. Além disso, tanto as avaliações quanto as etapas da análise qualitativa (síntese das justificativas, indução de categorias e classificação) são conduzidas por LLMs, e a verificação manual compara a categoria atribuída com a justificativa sintetizada e o *reasoning* dos juízes, não diretamente com o código-fonte. O Gemini 3.5 Flash-Lite atua como juiz e também como sintetizador das justificativas, o que pode favorecer a sua própria leitura dos casos. Por fim, o juiz não recebe o esquema de entrada (*inputSchema*) que o agente recebe em tempo de execução, recorte que isola a descrição, como em Faghih et al. (2025), e que coincide com o *script* de avaliação de Hasan et al. (2026b). Parte dos parâmetros considerados não documentados pode, portanto, estar descrita nesse esquema, embora, na rubrica desses autores, parâmetros documentados apenas no esquema recebam a nota mínima em *Parameter Explanation*. Como Li et al. (2026) mostram que o esquema também influencia a escolha da ferramenta pelo modelo, os resultados deste estudo se referem à qualidade da descrição, e não à interface completa vista pelo agente.

Validade interna. O código fornecido ao avaliador é obtido por uma heurística de resolução de chamadas baseada em nome, limitada a três níveis de profundidade e a 32.000 caracteres (Seção 4.2). Basear-se no nome significa associar cada chamada a uma definição com o mesmo nome, sem verificar o tipo do objeto que a recebe, como faria

um compilador. Assim, quando duas classes definem métodos homônimos, o trecho de outra classe pode ser incluído por engano, e o juiz passa a comparar a descrição com um código que não é o da ferramenta; inversamente, chamadas não resolvidas e os limites de profundidade e tamanho podem deixar de fora parte da lógica relevante. Em ambos os casos, a mudança de nota pode refletir o código extraído, e não a ferramenta real, embora resoluções ambíguas sejam sinalizadas. As respostas dos juízes não são determinísticas, e, na verificação manual da classificação das justificativas, os dois avaliadores concordaram com 86% e 80% das categorizações, mas divergiram entre si em 30% dos casos. Por outro lado, a amostra de divergência confirmada exige que todos os juízes concordem que a nota mudou, o que a torna conservadora frente ao ruído de um juiz isolado.

Validade externa. A amostra restringe-se a 206 repositórios públicos do GitHub com ao menos 100 estrelas, concentrados em Python e TypeScript, e três repositórios respondem por 46,7% das ferramentas, o que pode enviesar resultados agregados. Os resultados também refletem três modelos específicos em uma versão e momento determinados; como os próprios juízes divergem na direção do efeito (RQ2), não é possível generalizar o comportamento observado para outros avaliadores.

7. Conclusão e Trabalhos Futuros

Este trabalho investigou se a avaliação da qualidade de descrições de ferramentas MCP é influenciada pela disponibilização do código da ferramenta ao avaliador. Para isso, 12.166 ferramentas de 206 repositórios de servidores MCP foram extraídas, junto com o código alcançado por sua árvore de chamadas, e avaliadas por três LLMs segundo a rubrica de seis componentes de Hasan et al. (2026b), com e sem acesso ao código.

Os resultados mostram que o acesso ao código altera a avaliação de forma estatisticamente significativa, embora em direção que depende do juiz. Na maior parte das ferramentas em que todos os juízes mudaram a nota (89,3%), a mudança decorre de um achado concreto no código, sobretudo parâmetros e opções não documentados, modos de falha não divulgados e detalhes da estrutura de saída. Isso mostra que o código dá ao avaliador acesso a deficiências que a descrição sozinha não revela, mas não permite afirmar que a avaliação com código seja mais precisa, o que exigiria um julgamento humano de referência. A direção do efeito, por outro lado, depende do avaliador: o DeepSeek tende a reduzir as notas, enquanto o Gemini e o Qwen3 tendem a elevá-las, mesmo quando o código revela omissões, porque creditam à descrição o que aprenderam pelo código; *Purpose* é o atributo com mais notas alteradas e *Guidelines*, o que mais muda de faixa, enquanto *Limitations* e *Examples* são os menos afetados. As diferenças entre os cenários são estatisticamente significativas nos seis atributos e, mais do que isso, relevantes em magnitude: em quatro deles, cerca de uma em cada três ferramentas muda de faixa de qualidade quando o avaliador tem acesso ao código, o suficiente para alterar quais descrições seriam consideradas adequadas ou deficientes.

Esses achados têm uma consequência que vai além da avaliação. O agente que consome uma ferramenta MCP decide quando e como invocá-la apenas a partir de sua descrição, e as informações que o código revelou e a descrição omitia (parâmetros aceitos, condições de erro, formato da saída e limites operacionais) são justamente as que orientam essas decisões. A qualidade da descrição, portanto, pode condicionar o comportamento do agente e o resultado que ele obtém, e uma avaliação que considere apenas o texto não

detecta omissões que podem levar o agente a errar. Ao mesmo tempo, a tendência de parte dos juízes a elevar as notas com o código aponta um risco: o de o avaliador atribuir à descrição aquilo que aprendeu pelo código, avaliando a ferramenta e não o texto que o agente efetivamente recebe.

Conclui-se que avaliar descrições MCP isoladamente deixa de captar uma parcela relevante de suas deficiências e que o código, quando disponível, pode ser usado para revelar o que a descrição omite, desde que o avaliador não o confunda com a própria descrição. Para quem constrói avaliadores automáticos, isso significa que fornecer o código não basta: é preciso instruir o avaliador a usá-lo apenas para verificar o texto, e escolher modelos que mantenham essa separação, já que juízes diferentes, diante da mesma rubrica, interpretam o papel do código de formas opostas. Para quem escreve descrições, as categorias mais frequentes indicam o que verificar antes de publicar uma ferramenta: os parâmetros aceitos, as condições de erro, o formato da saída e os limites operacionais. A principal contribuição do trabalho é demonstrar esse efeito em larga escala, com 12.166 ferramentas e três avaliadores, e oferecer uma caracterização empírica, em 16 categorias, do tipo de informação que o código revela, além de mostrar que a divergência entre avaliadores decorre do modo como cada um trata o código, e não do que nele encontra.

Como trabalhos futuros, propõe-se: (i) construir um padrão-ouro humano sobre uma amostra de ferramentas, permitindo medir diretamente se a avaliação com código é mais precisa e se a elevação de notas reflete uma descrição de fato melhor; (ii) verificar se as deficiências reveladas pelo código afetam o desempenho real de agentes, por exemplo em *benchmarks* baseados em execução como o MCP-Universe [Luo et al. 2025]; e (iii) usar o código para reescrever automaticamente as descrições e avaliar o ganho de qualidade resultante.

Referências

Anthropic (2026). Define tools. `https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools`. Acesso em: nov. 2026.

Avaya (2026). The Model Context Protocol: A Status Report for Enterprise Customer Experience Leaders. White paper, Avaya.

Borges, H., Hora, A., and Valente, M. T. (2016). Predicting the popularity of GitHub repositories. In *Proceedings of the 12th International Conference on Predictive Models and Data Analytics in Software Engineering*, pages 1–10.

Braun, V. and Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2):77–101.

Chaokromthong, K., Sintao, N., et al. (2021). Sample size estimation using Yamane and Cochran and Krejcie and Morgan and green formulas and Cohen statistical power analysis by G* Power and comparisions. *Apheit international journal of interdisciplinary social sciences and technology*, 10(2):76–86.

Faghih, K., Wang, W., Cheng, Y., Bharti, S., Sriramanan, G., Balasubramanian, S., Hosseini, P., and Feizi, S. (2025). Tool preferences in agentic LLMs are unreliable. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Pro-*

cessing, pages 20954–20969, Suzhou, China. Association for Computational Linguistics.

Femmer, H., Méndez Fernández, D., Wagner, S., and Eder, S. (2017). Rapid quality assurance with Requirements Smells. *Journal of Systems and Software*, 123:190–213.

GitHub (2026). REST API endpoints for search. `https://docs.github.com/en/rest/search/search`. Acesso em: nov. 2026.

Guo, H., Hao, Y., Zhang, Y., Xu, M., Lv, P., Chen, J., and Cheng, X. (2025). A Measurement Study of Model Context Protocol Ecosystem. *arXiv preprint arXiv:2509.25292*.

Han, J., Deng, S., Xia, X., Wang, D., and Yin, J. (2019). Characterization and prediction of popular projects on GitHub. In *Proceedings of the IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, volume 1, pages 21–26. IEEE.

Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., and Hassan, A. E. (2026a). Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.

Hasan, M. M., Li, H., Rajbahadur, G. K., Adams, B., and Hassan, A. E. (2026b). Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions. *arXiv preprint arXiv:2602.14878v3*.

Hsieh, H.-F. and Shannon, S. E. (2005). Three approaches to qualitative content analysis. *Qualitative Health Research*, 15(9):1277–1288.

ISO/IEC (2011). ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models. Technical report, International Organization for Standardization, Geneva, Switzerland.

ISO/IEC/IEEE (2017). ISO/IEC/IEEE 24765:2017 – Systems and software engineering – Vocabulary. Technical report, International Organization for Standardization, Geneva, Switzerland.

Khan, J. Y., Khondaker, M. T. I., Uddin, G., and Iqbal, A. (2021). Automatic Detection of Five API Documentation Smells: Practitioners' Perspectives. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 318–329.

Li, X., Qiao, F., and Gao, X. (2026). Which metadata matters for LLM tool selection in MCP? Evidence from controlled factor-level interventions. *arXiv preprint arXiv:2610.14334*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Luo, Z., Shen, Z., Yang, W., Zhao, Z., Jwalapuram, P., Saha, A., Sahoo, D., Savarese, S., Xiong, C., and Li, J. (2025). MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers. *arXiv preprint arXiv:2508.14704*.

Model Context Protocol (2026). Specification 2026-07-28: Tools. https://modelcontextprotocol.io/specification/2026-07-28/server/tools. Acesso em: nov. 2026.

National Security Agency (2026). Model Context Protocol (MCP): Security Design Considerations for AI-Driven Automation. Cybersecurity report, NSA Cybersecurity Information.

Pressman, R. S. and Maxim, B. R. (2016). *Engenharia de Software: Uma Abordagem Profissional*. AMGH, Porto Alegre, 8 edition.

Shi, Y., Zhang, X., Zhang, X., Shen, X., Ouyang, H., Qiu, H., Zhang, M., and Yang, M. (2026). Description-Code Inconsistency in Real-world MCP Servers: Measurement, Detection, and Security Implications. *arXiv preprint arXiv:2606.04769*.

Taraghi, M., Morovati, M. M., and Khomh, F. (2026). Real Faults in Model Context Protocol (MCP) Software: a Comprehensive Taxonomy. *arXiv preprint arXiv:2603.05637*.

Wilcoxon, F. (1945). Individual Comparisons by Ranking Methods. *Biometrics Bulletin*, 1(6):80–83.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J., and Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

Apêndice

A. Prompt dos Juízes

O *prompt* de sistema abaixo é o mesmo para os três juízes e para os dois cenários; o cenário é definido apenas pela presença do campo `SOURCE_CODE` no objeto JSON enviado como mensagem do usuário, que contém também os campos `name`, `server_name` e `description`.

```
1 You are an expert evaluator of Model Context Protocol (MCP) tool
      descriptions, participating in a research study on tool-
     description quality.
2
3 You will be given a JSON object describing one MCP tool, with
     the following fields:
4 - "name": the tool's identifier.
5 - "server_name": the MCP server (repository) the tool belongs to
     .
6 - "description": the natural-language description exposed to an
     LLM agent deciding whether and how to call the tool. This is
     what you are evaluating.
7 - "SOURCE_CODE" (optional): source code context for the tool,
     including its own implementation and up to two levels of the
     functions it calls. Only present in some evaluations.
8
```

```
Score the "description" against each of the following 6 components, independently, on a 5-point Likert scale. A low score on one component must NOT depress your score on another -- evaluate each on its own terms.

1. Purpose (purpose): How clearly and completely the description explains what the tool does.
2. Guidelines (guidelines): Whether the description gives both decision-making activation criteria (when to use the tool) and operational instructions (how to use it).
3. Limitations (limitations): Whether known constraints, caveats, or corner cases where the tool may fail are disclosed.
4. Parameter Explanation (parameter_explanation): Whether the roles of the tool's input parameters are explained beyond just their data types.
5. Length & Completeness (length_completeness): Whether the description reaches at least three to four sentences of substantive detail, rather than being a terse fragment.
6. Examples (examples): Whether illustrative examples of correct and effective usage are given.

Likert scale (apply uniformly to all 6 components):
5 = Ideal: fully satisfies this component, with no meaningful ambiguity.
4 = Minor ambiguity: mostly satisfies this component; small gaps a reader could resolve unaided.
3 = Minimum viable: meets a bare threshold for this component; noticeably incomplete but usable.
2 = Vague: this aspect is present but too thin or unclear to rely on.
1 = Missing: this aspect is absent from the description entirely.

Handling "SOURCE_CODE":
When source code snippets are provided, the tool should first be evaluated independently,
without considering the source code. After this initial evaluation, the source code should
be taken into account to identify any inconsistencies between the tool's description and
its actual implementation. Based on these findings, the initial score may be adjusted accordingly.
```

Listing 1. Prompt de sistema – avaliação das descrições pelos juízes

B. Prompts Utilizados na Análise Qualitativa das Divergências

Este apêndice reproduz, na íntegra e no idioma original (inglês, mesmo idioma em que a justificativa de cada juiz é produzida), os três *prompts* de sistema (*system instruction*) utilizados nas três etapas da análise qualitativa descrita na Seção 4.4. Em todos os casos,

o modelo recebe o *prompt* de sistema reproduzido abaixo e, na mensagem do usuário, o objeto ou lista JSON de entrada descrito no próprio texto do *prompt*; a saída é restrita a um objeto ou lista JSON, sem texto adicional, por meio do mecanismo de saída estruturada do respectivo modelo: Gemini nas Etapas 1 e 3, e DeepSeek na Etapa 2.

Etapa 1: Geração de Justificativas

Aplicado individualmente a cada ferramenta da amostra de divergência confirmada (Seção 4.3), recebendo como entrada um objeto JSON com os campos `tool_uid`, `tool_name`, a lista `avaliacoes` (uma entrada por juiz e cenário, já restrita aos componentes da rubrica cuja nota divergiu) e `source_code`.

```
1 You are a research assistant helping analyze results from an
     undergraduate thesis ("Model Context Protocol (MCP): Avaliaçã
     o da Qualidade de Descrições de Tools com Base em Contexto de
      Código-Fonte") that studies whether giving an LLM-as-judge
     access to a tool's source code changes its evaluation of the
     tool's natural-language description. Several judge models
     independently scored the SAME description on a 1-5 Likert
     scale across 6 rubric components (Purpose, Guidelines,
     Limitations, Parameter Explanation, Length & Completeness,
     Examples), once without the source code ("description_only")
     and once with it ("with_source"). The judges were instructed
     to first score the description without considering the source
      code, and then to use the source code to identify
     inconsistencies between the description and the
     implementation, adjusting the initial score accordingly. The
     instruction does not restrict the direction of that
     adjustment, so a score change may or may not be grounded in a
      concrete finding from the code.
2
3 You will receive one JSON object describing a single tool for
     which at least one component's score changed between the two
     scenarios, for every judge that evaluated it unanimously. It
     has:
4 - "tool_uid": a unique identifier for the tool.
5 - "tool_name": the tool's name.
6 - "avaliacoes": a list of per-judge, per-scenario entries. Each
     has "scenario" ("description_only" or "with_source"), "
     provider" (which judge model), and "scores" -- only the
     components whose score differs between the two scenarios for
     that judge, each with the numeric "score" and that judge's
     own "reasoning" explaining it.
7 - "source_code": the actual source snippet(s) the "with_source"
     judges saw.
8
9 Task: read every judge's reasoning for its changed component(s),
      cross-reference it against "source_code", and write ONE
     justification (3-5 sentences) explaining WHAT the source code
      revealed that the plain-text description did not -- e.g. an
```

```
    undocumented parameter, an undisclosed behavior/output, a
    missing limitation or error case, or a direct contradiction
    between the description and the implementation. If judges
    disagree on the reason, or if a judge's reasoning gives no
    concrete evidence from the code (just a vaguer/stricter
    reading of the same text), say so plainly instead of
    inventing a justification.
10
11  Ground every claim in specific details from the reasoning and/or
     the source code. Do not just restate "the score changed" or
    copy one judge's reasoning verbatim -- synthesize across
    judges. If different judges changed different components for
    different reasons, address each briefly, in one paragraph.
    Always write the justification in English, regardless of the
    reasoning's original language (it is already in English in
    this dataset, but do not drop this rule if that ever changes)
    .
12
13  Be conservative per component: a strong, well-evidenced finding
    for one judge/component (e.g. a parameter revealed to be
    hardcoded, or a disclosed error case) must NOT be stretched
    to also explain a different judge's or a different component'
    s change unless that other reasoning independently supports
    it. When a specific score change's reasoning is vague or
    generic (e.g. "now seems clearer," "provides more context,"
    with no concrete detail cited), say explicitly that no clear
    code-based reason is evident for that particular change
    instead of inferring one. Never introduce terminology, named
    standards, formulas, or mechanisms that do not literally
    appear in the reasoning or in "source_code" -- even when they
     would be plausible, typical domain knowledge for a tool like
     this one. If you are not sure a claim is directly supported
    by the given text, leave it out.
14
15  Output ONLY one JSON object, no text before or after it, in
    exactly this shape:
16  {"tool_uid": "<the tool_uid from the input, verbatim>", "
    justification": "<your justification>"}
```

Listing 2. Prompt de sistema – geração de justificativas

Etapa 2: Identificação Indutiva de Categorias

Aplicado uma única vez, recebendo como entrada o conjunto completo de justificativas produzidas na Etapa 1, no formato de uma lista JSON de objetos {"tool_uid": ..., "justification": ...}.

```
1  You are a qualitative research assistant conducting a thematic
    analysis for an undergraduate thesis ("Model Context Protocol
     (MCP): Avaliação da Qualidade de Descrições de Tools com
    Base em Contexto de Código-Fonte") that studies whether
```

```
    giving an LLM-as-judge access to a tool's source code changes
     its evaluation of the tool's natural-language description.
    For every tool where every judge that evaluated it changed at
     least one rubric score between the "description_only" and "
    with_source" scenarios, a short English justification was
    already written, explaining WHAT the source code revealed
    that the plain-text description did not.
2
3 You will receive a JSON array of {"tool_uid": ..., "
    justification": ...} objects -- one per tool, every
    justification independently written from that tool's own
    judge reasoning and source code.
4
5 Task: perform an inductive thematic analysis (open coding,
    following Braun & Clarke's six-phase method: familiarization,
     initial coding, searching for themes, reviewing themes,
    defining and naming themes) across ALL justifications to
    identify the recurring categories of WHAT KIND of thing the
    source code tends to reveal that the description does not. Do
     not start from a predefined list -- let the categories
    emerge bottom-up from patterns across the justifications
    themselves. Each category must describe a MECHANISM (what the
     code exposed), never a rubric component (Purpose, Guidelines
    , Limitations, Parameter Explanation, Length & Completeness,
    Examples) and never a direction (score went up/down) -- e.g.
    "the code reveals a hardcoded parameter the description
    presents as configurable" is a mechanism; "Limitations score
    decreased" is not.
6
7 Requirements for the final set of categories:
8 - Between 5 and 15 categories. Fewer undersegments genuinely
    distinct patterns; more usually means two categories should
    be merged, or a category is really a one-off outlier rather
    than a recurring pattern.
9 - Each category must be grounded in multiple justifications
    across different tools -- never create a category for a
    single idiosyncratic case.
10 - Categories must be mutually distinguishable: if two candidate
    categories would classify the same justification the same way
     almost every time, merge them into one.
11 - Name each category concisely (2-5 words, a short analytic
    label, Title Case).
12 - Each description must state, in 2-3 sentences: (a) what the
    category captures, in terms of the specific kind of
    discrepancy or disclosure found in the code, and (b) one
    illustrative, paraphrased example pattern drawn from the
    justifications -- never quote or name a specific tool_uid.
13
```

```
Output ONLY a JSON array, no text before or after it, in exactly this shape:
[{"category": "<Title Case name>", "description": "<2-3 sentence description>"}, ...]

Write everything in English.
```

Listing 3. Prompt de sistema – identificação indutiva de categorias

Etapa 3: Classificação Dedutiva por Categoria

Aplicado individualmente a cada ferramenta da amostra de divergência confirmada. O trecho reproduzido como `<lista de categorias da Etapa 2, uma por linha>` é substituído, em tempo de execução, pela lista de categorias produzida na Etapa 2 (nome e descrição de cada uma); `"none"` é a categoria de reserva explícita, utilizada quando nenhuma das categorias da lista fixa corresponde à justificativa analisada.

```
You are a qualitative research assistant performing directed content analysis for an undergraduate thesis ("Model Context Protocol (MCP): Avaliação da Qualidade de Descrições de Tools com Base em Contexto de Código-Fonte") that studies whether giving an LLM-as-judge access to a tool's source code changes its evaluation of the tool's natural-language description.

A prior inductive thematic analysis identified the following FIXED set of categories, describing the MECHANISM by which the source code revealed something the description did not. Treat this list as closed -- do not invent new categories, rename any, or merge/split any, even if a justification seems to need a slightly different label:

<lista de categorias da Etapa 2, uma por linha>

You will receive one JSON object: {"tool_uid": ..., "justification": ...}. Assign every category above whose description genuinely matches what THIS justification says -- it may match more than one category (do not force a single label), and should match zero categories only if none of them genuinely fits (in that case, output the literal string "none" as the sole entry, never an empty list).

Base your assignment only on this justification's own content, not on any other tool. Output ONLY one JSON object, no text before or after it, in exactly this shape:
{"tool_uid": "<copied verbatim from the input>", "categories": ["<matching category name(s), or \"none\">"]}
```

Listing 4. Prompt de sistema – classificação dedutiva por categoria

C. Categorias de Mecanismo

A Tabela 3 apresenta as 16 categorias de mecanismo produzidas pela análise temática indutiva (Seção 4.4), cujas frequências estão na Tabela 1.

Tabela 3. Categorias de mecanismo identificadas pela análise temática indutiva sobre a amostra de divergência confirmada

Categoria	Descrição
Superfície de Entrada Não Documentada	A assinatura ou o esquema de entrada do código aceita parâmetros, filtros, identificadores de sessão ou combinações obrigatórias que a descrição nunca menciona.
Modos de Falha e Erros Não Divulgados	Tratamentos de erro, exceções, falhas de dependência e mensagens de estado vazio da implementação não são antecipados pela descrição.
Estrutura Concreta da Saída	A estrutura exata do retorno (campos, aninhamento, formatação, variações por sucesso/falha) é mais específica do que a descrição sugere.
Regras de Validação e Contratos de Formato Não Divulgados	Conjuntos de valores permitidos, formatos, regras de exclusão mútua ou campos obrigatórios fixados no código não são declarados na descrição.
Limites Operacionais, Padrões e Paginação Não Divulgados	Tetos, truncamentos, *timeouts*, paginação e valores padrão não declarados são impostos pela implementação sem menção na descrição.
Contexto de Fluxo e Ativação	O código revela pré-requisitos de estado de sessão, ordem de chamadas entre ferramentas irmãs, ou contexto ambiente que a descrição não articula.
Escopo de Processamento e Mecânica Interna Subestimados	A implementação realiza bem mais processamento ou mecânica interna (múltiplas sub-chamadas, *cache*, chamadas de rede, lógica assíncrona) do que a descrição sugere.
Confirmação de Encapsulamento Trivial	O código é um encaminhamento mínimo cujo comportamento já correspondia exatamente ao texto, sem revelar nada novo.
Valores Fixos e Padrões Divergentes	Um valor apresentado como configurável está, na realidade, fixado no código, ou o valor padrão documentado diverge do valor efetivamente usado.
Dependências e Pré-requisitos de Ambiente Ocultos	Serviços externos, variáveis de ambiente, credenciais, binários ou configurações exigidos pelo código não são mencionados na descrição.
Semântica de Correspondência e Normalização de Entrada	O código processa entradas de forma não óbvia (correspondência por substring, normalização de apelidos, resolução difusa de nomes) sem que a descrição mencione essa semântica.
Controles de Acesso, Papel e Autorização	Permissões, autenticação, verificações de papel/administrador e outras restrições de acesso aplicadas no código estão ausentes da descrição.
Parâmetros Fantasmas ou Divergentes	A descrição documenta parâmetros que o código não possui, ou que foram renomeados, removidos ou substituídos na implementação.
Funcionalidade Anunciada Não Implementada	O código expõe um *stub*, um retorno fixo, ou simplesmente não implementa a funcionalidade que a descrição promete.
Sistema-Alvo Divergente	O código revela uma plataforma, formato de arquivo ou granularidade de recurso diferente da que a descrição declara.
Efeitos Colaterais Não Divulgados	O código produz efeitos colaterais (exclusões em cascata, escrita de arquivos, notificações) além da ação anunciada pela descrição.